

Fase 1: Fundamentos Teóricos e Nivelamento

Módulo 1: Fundamentos de IA

1. Teoria

- **Workloads de IA:** Uma carga de trabalho (*workload*) de IA é o conjunto de tarefas de computação necessárias para treinar ou executar um modelo. Divide-se em:
 - **Training (Treino):** Processo intensivo em computação (geralmente GPUs/TPUs) para ajustar os pesos de uma rede neural com base num volume massivo de dados.
 - **Inference (Inferência):** Fase de produção na qual o modelo já treinado recebe novos dados de entrada e gera uma predição/resposta. Requer otimização de latência e custo.
- **Paradigma do Machine Learning (ML):** Diferente da programação tradicional (onde programadores escrevem regras exatas para processar dados), no ML os algoritmos consomem **dados de entrada** e **respostas desejadas** para aprender as regras e padrões subjacentes.
- **Processamento de Linguagem Natural (NLP):** Ramo da IA focado na interação entre computadores e linguagem humana. Evoluiu de abordagens estatísticas/baseadas em regras (TF-IDF, n-grams) para modelos sequenciais (RNNs, LSTMs) e, atualmente, para arquiteturas baseadas em *Transformers*.
- **Visão Computacional (CV):** Campo dedicado ao processamento e extração de informação a partir de imagens e vídeos. Envolve tarefas como classificação de imagens, detecção de objetos (ex.: YOLO), segmentação semântica e extração de *features* visuais.
- **IA Generativa (GenAI) vs. IA Discriminativa:**
 - **Discriminativa:** Modela $P(Y|X)$ — aprende a fronteira de decisão entre classes (ex.: "esta imagem é um cão ou um gato?").
 - **Generativa:** Modela a distribuição de probabilidade subjacente $P(X)$ ou $P(X|Y)$ — aprende a criar novas amostras de dados que se assemelham aos dados de treino (ex.: gerar um novo texto ou imagem).
- **Agentes de IA:** Sistemas autónomos que percebem o seu ambiente, tomam decisões com base em objetivos e executam ações utilizando ferramentas externas (*tools*).

2. Prática Mão na Massa (Python)

Este exemplo demonstra o ciclo completo de um *workload* simples: vetorização de texto (NLP clássico) e treino de um modelo discriminativo com a biblioteca scikit-learn.

```

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Conjunto de Dados (Workload de Treino)
corpus_treino = [
    "A IA generativa cria novos textos e imagens",
    "Modelos de machine learning aprendem com dados",
    "O servidor de inferência está com alta latência",
    "Visão computacional detecta objetos em vídeos",
    "Redes neurais otimizam tarefas complexas de NLP"
]
labels_treino = [1, 1, 0, 1, 1] # 1: Tópico de IA/ML, 0:
Infraestrutura/Outros

# 2. Pré-processamento e Extração de Features (NLP/Vectorization)
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(corpus_treino)

# 3. Treino do Modelo (Training Phase)
model = LogisticRegression()
model.fit(X_train, labels_treino)

# 4. Inferência (Inference Phase)
novos_dados = [
    "Agentes de IA tomam decisões autônomas",
    "O banco de dados do servidor caiu"
]
X_test = vectorizer.transform(novos_dados)
predicoes = model.predict(X_test)

# Exibição dos resultados de inferência
for texto, pred in zip(novos_dados, predicoes):
    categoria = "IA/ML" if pred == 1 else "Infraestrutura"
    print(f"Texto: '{texto}' -> Classificação: {categoria}")

```

Módulo 2: Matemática Intuitiva para IA

1. Teoria

- **Vetores e Espaços Vetoriais:** Um vetor é um *array* ordenado de números que representa um ponto num espaço multidimensional. Em IA, dados (textos, imagens, áudios) são convertidos em vetores de *features* ou *embeddings*.
- **Matrizes e Operações:** Estruturas bidimensionais de números. A multiplicação matricial é a operação fundamental nas redes neurais, representando transformações lineares dos dados de entrada.

- **Métricas de Distância:**

- **Distância Euclidiana (L_2):** Mede a menor distância em linha reta entre dois pontos num espaço n -dimensional:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{i=1}^n (u_i - v_i)^2}$$

- **Similaridade de Cosseno:** Mede o cosseno do ângulo entre dois vetores. Útil para NLP/RAG pois ignora a magnitude e foca na orientação espacial (direção do significado):

$$\text{Sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

- **Gradiente e Otimização Conceptual:**

- **Gradiente (∇f):** Vetor de derivadas parciais que indica a direção de maior crescimento de uma função de perda (*loss function*).
- **Gradient Descent (Descida do Gradiente):** Algoritmo de otimização onde atualizamos os pesos do modelo no sentido **oposto** ao gradiente para minimizar o erro:

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t)$$

onde α é a taxa de aprendizagem (*learning rate*).

2. Prática Mão na Massa (Python / NumPy)

```
import numpy as np

# 1. Vetores de Embeddings fictícios (3 dimensões)
vetor_rei = np.array([0.50, 0.80, 0.12])
vetor_rainha = np.array([0.48, 0.82, 0.15])
vetor_carro = np.array([0.01, 0.05, 0.99])

# 2. Função para cálculo da Similaridade de Cosseno
def cosine_similarity(u, v):
    dot_product = np.dot(u, v)
    norm_u = np.linalg.norm(u)
    norm_v = np.linalg.norm(v)
    return dot_product / (norm_u * norm_v)

sim_rei_rainha = cosine_similarity(vetor_rei, vetor_rainha)
sim_rei_carro = cosine_similarity(vetor_rei, vetor_carro)

print(f"Similaridade Cosseno (Rei, Rainha): {sim_rei_rainha:.4f}")
print(f"Similaridade Cosseno (Rei, Carro): {sim_rei_carro:.4f}")

# 3. Simulação de um passo de Gradient Descent (Otimização)
```

```
# Função de Perda fictícia:  $L(w) = w^2 \rightarrow$  Gradiente  $dL/dw = 2*w$ 
w = 10.0          # Peso inicial
learning_rate = 0.1

print(f"\nPeso inicial (w): {w}")
for passo in range(1, 6):
    gradiente = 2 * w
    w = w - learning_rate * gradiente
    erro = w**2
    print(f"Passo {passo}: Gradiente = {gradiente:.2f} | Novo Peso w = {w:.4f} | Perda = {erro:.4f}")
```

Módulo 3: Estatística para IA

1. Teoria

- **Probabilidade e Variáveis Aleatórias:** Base para lidar com a incerteza nos dados.
 - **Probabilidade Condicional:** $P(A|B)$ representa a probabilidade do evento A ocorrer dado que o evento B já ocorreu. Base do Teorema de Bayes, essencial para classificadores probabilísticos.
- **Distribuições de Probabilidade:**
 - **Distribuição Normal (Gaussiana):** Caracterizada por uma curva em forma de sino definida pela média (μ) e desvio padrão (σ). É a premissa central de muitos algoritmos de aprendizado supervisionado e processamento de sinal.
 - **Distribuições Bernoulli/Binomial:** Modelam eventos binários (sucesso/fracasso).
- **Testes de Hipótese:** Procedimento estatístico para validar se uma diferença observada nos dados é estatisticamente significativa ou fruto do acaso.
 - **Hipótese Nula (H_0):** Premissa de que não há efeito ou diferença.
 - **p-valor:** Probabilidade de obter um resultado igual ou mais extremo assumindo que H_0 é verdadeira. Se $p < 0.05$, rejeita-se H_0 .
- **Métricas Estatísticas Fundamentais:**
 - **Média (μ):** Medida de tendência central.
 - **Variância (σ^2):** Dispersão dos dados em relação à média.

- **Covariância e Correlação:** Medida da relação linear entre duas variáveis contínuas (ex.: Pearson r).

2. Prática Mão na Massa (Python / SciPy / Matplotlib)

```
import numpy as np
import scipy.stats as stats

# Configuração de semente para reproducibilidade
np.random.seed(42)

# 1. Geração de duas amostras (ex.: tempo de resposta de dois modelos de IA)
modelo_A = np.random.normal(loc=120, scale=15, size=100) # Média = 120ms
modelo_B = np.random.normal(loc=112, scale=15, size=100) # Média = 112ms

# 2. Estatística Descritiva
print(f"Modelo A - Média: {np.mean(modelo_A):.2f}ms | Desvio Padrão: {np.std(modelo_A):.2f}")
print(f"Modelo B - Média: {np.mean(modelo_B):.2f}ms | Desvio Padrão: {np.std(modelo_B):.2f}")

# 3. Teste de Hipótese (t-test de duas amostras independentes)
# H0: A latência média do Modelo A e do Modelo B é IGUAL.
# H1: A latência média do Modelo B é estatisticamente menor/diferente.
t_stat, p_value = stats.ttest_ind(modelo_A, modelo_B)

print(f"\nEstatística t: {t_stat:.4f}")
print(f"p-valor: {p_value:.4e}")

alpha = 0.05
if p_value < alpha:
    print("Conclusão: Rejeitamos a hipótese nula (H0). A diferença de latência entre os modelos é estatisticamente significativa.")
else:
    print("Conclusão: Não há evidência suficiente para rejeitar H0. A diferença pode ser fruto do acaso.")
```